Министерство образования и исследований
Государственный университет Молдовы
Отдел математики и информатики

ОТЧЕТ

по лабораторной работе №3
«Основы работы с массивами, функциями и объектами в JavaScript»

Выполнил: студент гр. IA2503ru
Б. Антонович
Проверил: Н. Калин

Кишинёв 2026

1. Теоретическая часть

1.1. Формулировка задачи

Цель лабораторной работы — создать консольное приложение на JavaScript для анализа финансовых транзакций. Приложение должно содержать массив объектов-транзакций и набор функций для их обработки: фильтрации, подсчёта сумм, поиска, группировки и анализа данных.

Каждый объект транзакции содержит следующие поля: transaction_id (уникальный идентификатор), transaction_date (дата), transaction_amount (сумма), transaction_type (тип: debit или credit), transaction_description (описание), merchant_name (название магазина), card_type (тип карты).

1.2. Описание цели и основные этапы работы

Цель работы — изучить основы работы с массивами и функциями в JavaScript, применяя их для обработки и анализа данных.

Основные этапы выполнения работы:

- Шаг 1 — создание файла main.js и массива транзакций с тестовыми данными.
- Шаг 2 — реализация функций анализа: поиск уникальных типов, подсчёт сумм, фильтрация по типу/дате/магазину/диапазону, поиск по ID, анализ активности по месяцам.
- Шаг 3 — тестирование всех функций на полном массиве, пустом массиве и массиве с одной транзакцией, вывод результатов в консоль.

2. Практическая часть

2.1. Особенности реализации

Работа выполнена в одном файле main.js. Ниже описаны наиболее интересные моменты реализации.

Уникальные типы транзакций через Set

Функция getUniqueTransactionTypes использует объект Set, который автоматически исключает дубликаты. Массив типов получается через map(), затем оборачивается в Set и разворачивается обратно в массив через spread-оператор:

```
function getUniqueTransactionTypes(transactions) {
  return [...new Set(transactions.map(t => t.transaction_type))];
}
```

Рисунок 1 — Функция getUniqueTransactionTypes в коде

Фильтрация по дате с необязательными параметрами

Функция calculateTotalAmountByDate принимает year, month и day как необязательные параметры. Фильтрация происходит только по тем параметрам, которые переданы. Это реализовано через цепочку условий с оператором && внутри filter():

```
function calculateTotalAmountByDate(transactions, year, month, day) {
  return transactions
    .filter(t => {
      const date = new Date(t.transaction_date);
      if (year && date.getFullYear() !== year) return false;
      if (month && date.getMonth() + 1 !== month) return false;
      if (day && date.getDate() !== day) return false;
      return true;
    })
    .reduce((sum, t) => sum + t.transaction_amount, 0);
}
```

Рисунок 2 — Функция calculateTotalAmountByDate в коде

Сравнение дат в строковом формате

Для функций getTransactionsInDateRange и getTransactionsBeforeDate сравнение дат выполняется напрямую через операторы >= и <. Это работает корректно, потому что даты хранятся в формате ISO 8601 (YYYY-MM-DD), и лексикографическое сравнение строк совпадает с хронологическим:

```
function getTransactionsInDateRange(transactions, startDate, endDate) {
  return transactions.filter(t => t.transaction_date >= startDate && t.transaction_date <= endDate);
}
```

Рисунок 3 — Функция getTransactionsInDateRange в коде

Подсчёт транзакций по месяцам

Функции findMostTransactionsMonth и findMostDebitTransactionMonth формируют объект-счётчик, где ключом является строка вида YYYY-MM (первые 7 символов даты). После накопления данных через forEach, самый активный месяц определяется с помощью Object.keys и reduce:

```
function findMostTransactionsMonth(transactions) {
  const counts = {};
  transactions.forEach(t => {
    const month = t.transaction_date.slice(0, 7);
    counts[month] = (counts[month] || 0) + 1;
  });
  return Object.keys(counts).reduce((a, b) => counts[a] > counts[b] ? a : b);
}
```

Рисунок 4 — Функция findMostTransactionsMonth в коде

Обработка пустого массива

Функция calculateAverageTransactionAmount содержит явную проверку на пустой массив, чтобы избежать деления на ноль. Для остальных функций, построенных на filter() и reduce(), пустой массив обрабатывается автоматически — они возвращают пустой массив или 0 соответственно.

2.2. Ход работы

Шаг 1. Создание массива транзакций

Создан файл main.js. В нём объявлен массив transactions из 7 объектов. Каждый объект содержит все обязательные поля: transaction_id, transaction_date, transaction_amount, transaction_type, transaction_description, merchant_name, card_type. Транзакции охватывают три месяца (январь — март 2024) и включают как дебетовые, так и кредитовые операции.

Шаг 2. Реализация функций

Реализованы все 15 функций, описанных в задании, включая дополнительную функцию calculateTotalAmountByDate. Каждая функция задокументирована с помощью JSDoc-комментариев, указывающих типы входных параметров и возвращаемое значение.

Шаг 3. Тестирование

В конце файла выполняется вызов каждой функции с выводом результата в консоль через console.log. Тестирование проводилось в трёх сценариях: на полном массиве из 7 транзакций, на пустом массиве [] и на массиве с одной транзакцией. Результаты вывода консоли приведены на Рисунке 1.

```
  C:\Program Files\nodejs\node.exe .\main.js
> Unique types: (2) ['debit', 'credit']                                                    main.js:226
  Total amount: 1395                                                                       main.js:227
  Total by date (2024, Jan): 350                                                           main.js:228
> Debit transactions: (5) [{…}, {…}, {…}, {…}, {…}]                                       main.js:229
> Date range (2024-01-01 to 2024-02-28): (4) [{…}, {…}, {…}, {…}]                         main.js:230
> By merchant SuperMart: (2) [{…}, {…}]                                                   main.js:231
  Average amount: 199.28571428571428                                                       main.js:232
> Amount range 100-300: (4) [{…}, {…}, {…}, {…}]                                          main.js:233
  Total debit: 1075                                                                        main.js:234
  Most transactions month: 2024-03                                                         main.js:235
  Most debit month: 2024-03                                                                main.js:236
  Most transaction type: debit                                                             main.js:237
> Before 2024-03-01: (4) [{…}, {…}, {…}, {…}]                                             main.js:238
> Find by id 3: {transaction_id: '3', transaction_date: '2024-02-05', transaction_amount: 50, transaction_type: 'deb  …js:239
  it', transaction_description: 'Coffee and snacks', …}
> Descriptions: (7) ['Grocery shopping', 'Electronics purchase', 'Coffee and snacks', 'Clothing purchase', 'Restaura ….js:240
  nt dinner', 'Pharmacy', 'Online shopping']
                                                                                           main.js:242
  --- Empty array tests ---                                                                main.js:242
> Unique types: (0) []                                                                     main.js:243
  Total amount: 0                                                                          main.js:244
  Average amount: 0                                                                        main.js:245
  Most transaction type: equal                                                             main.js:246
                                                                                           main.js:249
  --- Single transaction tests ---                                                         main.js:249
> Unique types: (1) ['debit']                                                              main.js:250
  Total amount: 150                                                                        main.js:251
  Average amount: 150                                                                      main.js:252
  Most transaction type: debit                                                             main.js:253
```

Рисунок 5 — Вывод программы в консоль (фрагмент)

3. Вывод

В ходе лабораторной работы было создано консольное приложение на JavaScript для анализа финансовых транзакций. Были изучены и применены основные методы работы с массивами: map(), filter(), reduce(), forEach(), find(), а также объект Set для получения уникальных значений.

Все 15 функций успешно реализованы и протестированы на трёх наборах данных: полном массиве, пустом массиве и массиве с одной транзакцией. Особый интерес представляла реализация функции с необязательными параметрами и поиск наиболее активного месяца через объект-счётчик.

Ссылка на репозиторий GitHub: https://github.com/username/lab3-js-transactions

4. Ответы на контрольные вопросы

1. Какие методы массивов можно использовать для обработки объектов в JavaScript?

Для обработки массивов объектов в JavaScript наиболее часто используются следующие методы:

- map() — создаёт новый массив, применяя функцию к каждому элементу. Использован в mapTransactionDescriptions для получения массива описаний.
- filter() — создаёт новый массив из элементов, прошедших проверку. Использован в getTransactionByType, getTransactionsInDateRange и других функциях.
- reduce() — сворачивает массив в одно значение. Использован в calculateTotalAmount для суммирования.
- find() — возвращает первый элемент, удовлетворяющий условию. Использован в findTransactionById.
- forEach() — перебирает элементы без возврата нового массива. Использован при подсчёте транзакций по месяцам.
- Set — не метод массива, но позволяет легко получить уникальные значения. Использован в getUniqueTransactionTypes.

2. Как сравнивать даты в строковом формате в JavaScript?

Если даты хранятся в формате ISO 8601 (YYYY-MM-DD), их можно сравнивать напрямую как строки с помощью операторов <, >, <=, >= и ===. Это работает, потому что лексикографический порядок строк в данном формате совпадает с хронологическим порядком: более ранняя дата всегда лексикографически меньше более поздней.

Альтернативный способ — преобразование в объект Date через new Date(dateString) и сравнение числовых значений через getTime(). Это более надёжно для нестандартных форматов, но требует больше кода. В данной лабораторной работе использовался первый способ (строковое сравнение), так как все даты хранятся в формате YYYY-MM-DD.

3. В чём разница между map(), filter() и reduce() при работе с массивами объектов?

map() применяет функцию к каждому элементу и возвращает новый массив той же длины с преобразованными значениями. Исходный массив не изменяется. Пример использования: получить массив только из сумм транзакций.

filter() проверяет каждый элемент на соответствие условию и возвращает новый массив, содержащий только подходящие элементы. Длина нового массива может быть меньше или равна исходной. Пример использования: отобрать только дебетовые транзакции.

reduce() обходит массив и накапливает результат в одном значении (аккумуляторе). Возвращает не массив, а единственное значение — число, строку, объект и т.д. Пример использования: подсчитать общую сумму всех транзакций.

5. Список использованных источников

- MDN Web Docs — Array. [Электронный ресурс]. URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Array
- MDN Web Docs — Set. [Электронный ресурс]. URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Set
- MDN Web Docs — Date. [Электронный ресурс]. URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Date
- JSDoc — документирование JavaScript кода. [Электронный ресурс]. URL: https://jsdoc.app/
- Задание к лабораторной работе №3, кафедра FAF, ТУМ, 2024.